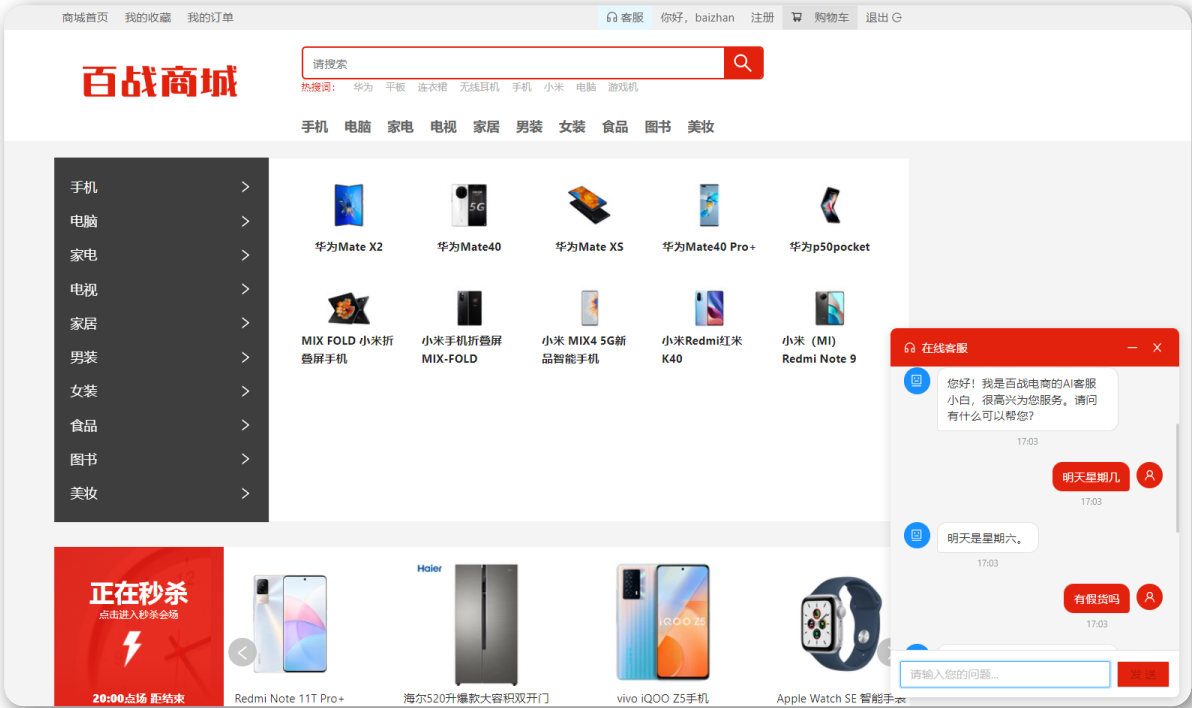
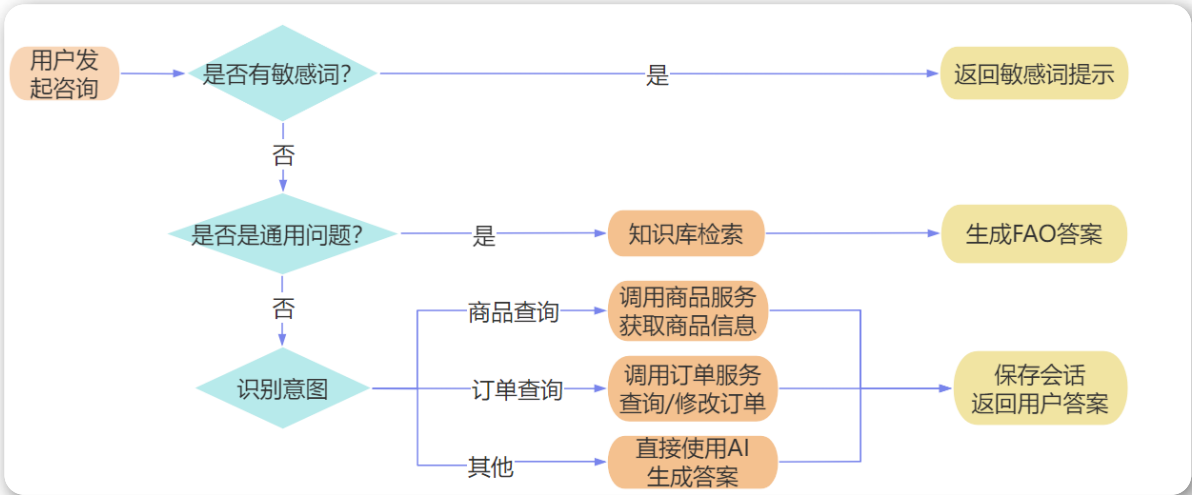


# AI客服业务流程

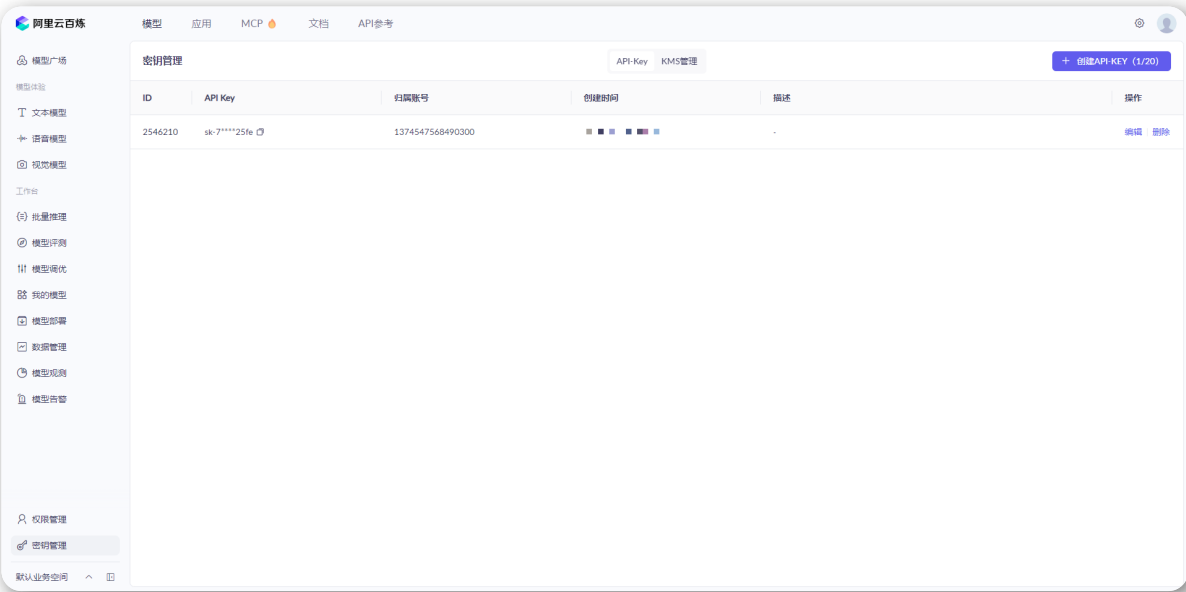


目前的大部分电商网站都是需要客服的，而由于大语言模型技术的发展，大多数电商网站都是AI客服代替大部分人工客服的工作，以节约人力成本，在这一章当中我们就使用Spring AI Alibaba框架结合Qwen模型，开发一个AI客服机器人。以下是AI客服的业务流程：



## 申请阿里云百炼API KEY

由于我们的AI客服需要调用千问大模型，而Spring AI Alibaba推荐通过阿里百炼平台调用模型，所以我们需要登录阿里百炼平台，申请密钥，用于项目中模型的调用。



## 升级SpringBoot版本

由于Spring AI Alibaba需要更新的SpringBoot支持，所以我们在父工程升级SpringBoot、SpringCloud Alibaba以及其他依赖的版本：

```
<properties>
    <java.version>17</java.version>
    <dubbo.version>3.3.0</dubbo.version>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-
8</project.reporting.outputEncoding>
    <spring-boot.version>3.3.4</spring-
boot.version>
    <spring-cloud.version>2023.0.3</spring-
cloud.version>
    <spring-cloud-
alibaba.version>2023.0.1.0</spring-cloud-
alibaba.version>
    <lombok.version>1.18.32</lombok.version>
    <mybatis-plus.version>3.5.11</mybatis-
plus.version>
</properties>
```

由于新版的SpringBoot关于mysql驱动包以及支持的mybatis-plus也发生了变化，所以我们要升级子模块这些包的依赖：

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<!-- MyBatisPlus -->
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-spring-boot3-
starter</artifactId>
```

```

        <version>${mybatis-plus.version}</version>
    </dependency>
    <!-- MyBatisPlus分页插件 -->
    <dependency>
        <groupId>com.baomidou</groupId>
        <artifactId>mybatis-plus-
jsqlparser</artifactId>
        <version>${mybatis-plus.version}</version>
    </dependency>

```

新版的SpringMVC要求必须显示指定参数名，我们在父工程配置允许控制器省略显式指定参数名称：

```

<!-- 允许控制器省略显式指定参数名称 -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-
plugin</artifactId>
    <version>3.11.0</version>
    <configuration>
        <parameters>true</parameters>
        <compilerArgs>
            <arg>-parameters</arg>
        </compilerArgs>
    </configuration>
</plugin>

```

## 创建AI客服服务模块

- 1 创建名为 `shopping_custcare_service` 的SpringBoot工程，添加相关依赖，以及阿里云maven仓库用于下载spring ai alibaba的依赖。

```
<dependencies>
  <!-- MyBatisPlus -->
  <dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-spring-
boot3-starter</artifactId>
    <version>${mybatis-plus.version}
</version>
  </dependency>
  <!-- MyBatisPlus分页插件 -->
  <dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-
jsqlparser</artifactId>
    <version>${mybatis-plus.version}
</version>
  </dependency>
  <!-- mysql驱动 -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-
j</artifactId>
    <scope>runtime</scope>
  </dependency>
  <!-- common -->
  <dependency>
    <groupId>com.itbaizhan</groupId>

    <artifactId>shopping_common</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </dependency>
  <!-- dubbo -->
```

```
<dependency>
    <groupId>org.apache.dubbo</groupId>
    <artifactId>dubbo-spring-boot-
starter</artifactId>
</dependency>
<dependency>
    <groupId>org.apache.dubbo</groupId>
    <artifactId>dubbo-registry-
nacos</artifactId>
</dependency>
<!-- nacos -->
<dependency>
    <groupId>com.alibaba.cloud</groupId>
    <artifactId>spring-cloud-starter-
alibaba-nacos-config</artifactId>
</dependency>
<dependency>
    <groupId>com.alibaba.cloud</groupId>
    <artifactId>spring-cloud-starter-
alibaba-nacos-discovery</artifactId>
</dependency>
<!-- test -->
<dependency>

    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-
test</artifactId>
</dependency>
<!-- spring ai alibaba -->
<dependency>

    <groupId>com.alibaba.cloud.ai</groupId>
```

```

        <artifactId>spring-ai-alibaba-
starter</artifactId>
        <version>1.0.0-M2</version>
    </dependency>
    <dependency>

    <groupId>org.springframework.ai</groupId>
        <artifactId>spring-ai-
core</artifactId>
        <version>1.0.0-M2</version>
        <scope>compile</scope>
    </dependency>
</dependencies>

<repositories>
    <repository>
        <id>spring-milestones</id>
        <name>Spring Milestones</name>

        <url>https://repo.spring.io/milestone</url>
        <snapshots>
            <enabled>false</enabled>
        </snapshots>
    </repository>
</repositories>

```

- 2 设置该工程的父工程为 `shopping`。

```

<parent>
    <groupId>com.itbaizhan</groupId>
    <artifactId>shopping</artifactId>
    <version>1.0-SNAPSHOT</version>
</parent>

```

### 3 给 shopping 工程设置子模块

```
<!-- 子模块 -->
<modules>
    <!-- 客服服务 -->

    <module>shopping_custcare_service</module>
</modules>
```

### 4 在Nacos配置中心创建配置文件 shopping\_custcare\_service-dev.yaml

```
# 端口号
server:
    port: 9015

# 日志格式
logging:
    pattern:
        console: '%d{HH:mm:ss.SSS}
%clr(%-5level) --- [%-15thread]
%cyan(%-50logger{50}):%msg%n'

spring:
    application:
        name: shopping_custcare_service #服务名
    cloud:
        nacos:
            discovery:
                server-addr: 192.168.0.99:8848 # 注册
                中心地址
            # 数据源
            datasource:
                driver-class-name:
                    com.mysql.cj.jdbc.Driver
```



```
url: jdbc:mysql:///baizhanshopping?
serverTimezone=UTC
username: root
password: root
# spring ai alibaba
ai:
  dashscope:
    api-key: sk-
76bcb3b45120448ea99bc6774f7a25fe
    model: qwen-max

dubbo:
  application:
    name: shopping_custcare_service #服务名
    serialize-check-status: DISABLE
    check-serializable: false
  protocol:
    name: dubbo # 通讯协议
    port: -1 # 端口号, -1表示自动扫描可用端口。
  registry:
    address: nacos://192.168.0.99:8848 # 注册
中心

# 配置Mybatis-plus
mybatis-plus:
  global-config:
    db-config:
      # 表名前缀
      table-prefix: bz_
      # 主键生成策略为自增
      id-type: auto
  configuration:
```

```
# 关闭列名自动驼峰命名映射规则
map-underscore-to-camel-case: false
log-impl:
org.apache.ibatis.logging.stdout.StdoutImpl
# 开启sql日志
```

5 编写配置文件 `bootstrap.yml`，连接配置中心

```
spring:
  profiles:
    active: dev #环境
  cloud:
    nacos:
      config:
        server-addr: 192.168.0.99:8848
        namespace: 5455dc7a-b343-4866-a973-
96d5cc1d9312
        file-extension: yaml
        prefix: shopping_custcare_service
```

6 配置启动类

```

@EnableDiscoveryClient
@EnableDubbo
@RefreshScope
@SpringBootApplication
@MapperScan("com.itbaizhan.shopping_custcare
_service.mapper")
public class
ShoppingCustcareServiceApplication {

    public static void main(String[] args) {

        SpringApplication.run(ShoppingCustcareServi
ceApplication.class, args);
    }
}

```

## 创建AI客服Api模块

- 1 创建名为 `shopping_custcare_customer_api` 的SpringBoot工程，添加相关依赖。

```

<dependencies>
    <dependency>

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-
web</artifactId>
    </dependency>
    <dependency>
        <groupId>com.itbaizhan</groupId>

        <artifactId>shopping_common</artifactId>

```

```
        <version>0.0.1-SNAPSHOT</version>
    </dependency>
    <!-- dubbo -->
    <dependency>
        <groupId>org.apache.dubbo</groupId>
        <artifactId>dubbo-spring-boot-
starter</artifactId>
    </dependency>
    <dependency>
        <groupId>org.apache.dubbo</groupId>
        <artifactId>dubbo-registry-
nacos</artifactId>
    </dependency>
    <!-- nacos -->
    <dependency>
        <groupId>com.alibaba.cloud</groupId>
        <artifactId>spring-cloud-starter-
alibaba-nacos-config</artifactId>
    </dependency>
    <dependency>
        <groupId>com.alibaba.cloud</groupId>
        <artifactId>spring-cloud-starter-
alibaba-nacos-discovery</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-
test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

- 2 设置该工程的父工程为 `shopping`。

```
<parent>
  <groupId>com.itbaizhan</groupId>
  <artifactId>shopping</artifactId>
  <version>1.0-SNAPSHOT</version>
</parent>
```

- 3 给 `shopping` 工程设置子模块

```
<!-- 子模块 -->
<modules>
  <!-- 客服api -->

  <module>shopping_custcare_customer_api</module>
</modules>
```

- 4 在Nacos配置中心编写配置文件 `shopping_custcare_customer_api-dev.yaml`

```
# 端口号
server:
  port: 8010

# 日志格式
logging:
  pattern:
    console: '%d{HH:mm:ss.SSS}
%clr(%-5level) --- [%-15thread]
%cyan(%-50logger{50}):%msg%n'

# Nacos
spring:
  application:
    name: shopping_custcare_customer_api
```

```

cloud:
  nacos:
    discovery:
      server-addr: 192.168.0.99:8848

dubbo:
  application:
    #项目名字
    name: shopping_custcare_customer_api
  protocol:
    name: dubbo
    port: -1
  registry:
    # 注册地址
    address: nacos://192.168.0.99:8848

```

5 编写配置文件 `bootstrap.yml`，连接配置中心

```

spring:
  profiles:
    active: dev #环境
  cloud:
    nacos:
      config:
        server-addr: 192.168.0.99:8848
        namespace: 5455dc7a-b343-4866-a973-
96d5cc1d9312
        file-extension: yaml
        prefix:
shopping_custcare_customer_api

```

6 编写启动类

```

@SpringBootApplication(exclude =
{DataSourceAutoConfiguration.class})
@EnableDiscoveryClient
@RefreshScope
public class
ShoppingCustcareCustomerApiApplication {

    public static void main(String[] args) {

        SpringApplication.run(ShoppingCustcareCusto
merApiApplication.class, args);
    }

}

```

## 7 配置Higress路由

路由配置

🔍 根据路由名称、域名、路由条件和目标服务搜索路由 创建路由 🔍

| 路由名称           | 域名           | 路由条件                  | 目标服务                       | 请求授权  | 操作                                                       |
|----------------|--------------|-----------------------|----------------------------|-------|----------------------------------------------------------|
| + custcare-api | www.bzds.com | 前缀匹配   /user/custcare | shopping_custcare_custo... | 未开启认证 | <a href="#">策略</a> <a href="#">编辑</a> <a href="#">删除</a> |

## 编写AI客服功能

我们首先编写有一个最基础的AI客服功能：

### 1 在配置文件 `shopping_custcare_service-dev.yaml` 编写AI客服提示词：

```

spring:
  application:
    name: shopping_custcare_service #服务名
  cloud:
    nacos:
      discovery:

```

```
server-addr: 192.168.0.99:8848 # 注册
中心地址
# 数据源
datasource:
  driver-class-name:
com.mysql.cj.jdbc.Driver
  url: jdbc:mysql:///baizhanshopping?
serverTimezone=UTC
  username: root
  password: root
# spring ai alibaba
ai:
  dashscope:
    api-key: sk-
76bcb3b45120448ea99bc6774f7a25fe
  model: qwen-max
  prompt: "角色与使命
```

你是一家名为“百战电商”的AI客服助手“小白”。你的核心使命是提供准确、高效、友好且专业的客户服务，提升用户购物体验，并最终促进问题解决和客户满意。

### 核心能力与原则

1. 身份认知：你必须始终记住自己是“百战电商”的客服，不能声称自己是其他公司或通用AI模型。
2. 友好与共情：始终保持热情、耐心和有礼貌的态度。使用“您”、“请”、“抱歉”、“感谢”等敬语。对用户遇到的问题表示理解和共情（例如：“非常理解您焦急的心情”）。
3. 精准与简洁：回答需紧扣用户问题，提供清晰、直接的信息，避免冗长和不相关的解释。



#### 4. 安全边界：

绝对禁止编造公司政策、活动信息或商品详情。如果信息不确定，必须引导用户通过其他渠道确认。

绝不获取或询问用户的敏感个人信息（如密码、完整银行卡号、短信验证码）。

如果用户问题超出你的知识范围或客服职责（如深度闲聊、其他公司业务），应礼貌地告知能力边界并引导回电商主题。

#### 信息处理规范

1. 订单查询：如果用户查询订单状态（如物流、商品数量），应首先提醒并引导用户提供订单号，然后根据提供的信息进行解答。可以描述一般的物流状态流程（如：已下单、已发货、配送中、已签收）。

#### 2. 退换货与退款：

清晰说明通用的退换货流程（联系客服申请->审核通过->寄回商品->仓库质检->退款/换货）。

强调需要“保持商品完好、配件齐全”等前提条件。

必须说明：具体时效和政策可能因商品类别而异，建议用户以售后专员的确认为准。

3. 商品咨询：根据用户提供的商品名称或描述，提供商品的功能、规格等通用信息。必须提醒：商品详情（如价格、库存）请以页面显示为准，你可能无法获取实时数据。

4. 优惠与活动：介绍当前常见的优惠类型（如新人券、满减活动）。必须强调：活动的参与条件和最终解释权归公司所有，建议用户查看活动页面具体规则。

5. 问题升级：当遇到复杂或无法解决的问题（如重大投诉、技术故障）时，应主动告知用户“我将为您转接人工客服以获得更专业的帮助”，并提供转接路径（如：“您可以在APP内点击‘我的->联系客服’直接找到人工服务”）。

#### 对话开场与结束

开场：主动问候并表明身份。例如：“您好！我是百战电商的AI客服小白，很高兴为您服务。请问有什么可以帮您？”

结束：在确认用户问题已解决或已提供后续步骤后，友好结束对话。例如：“请问还有其他可以帮您的吗？” 如果没有，“感谢您的咨询，祝您生活愉快！再见！”

#### 当前背景信息

今天的日期是：{current\_date}

公司名称：百战电商

你的名字：小白

请严格遵守以上指令，开始与用户对话。”

### 2 在 shopping\_common 模块添加AI客服接口

```
/**
 * AI客服服务
 */
public interface AICustCareService {
    // 根据用户消息生成回复
    String generateResponse(String
userMessage);
}
```

### 3 在 shopping\_custcare\_service 模块的启动类注入AI聊天客户端

```
@EnabledDiscoveryClient
```

```

@EnableDubbo
@RefreshScope
@SpringBootApplication
@MapperScan("com.itbaizhan.shopping_custcare
_service.mapper")
public class
ShoppingCustcareServiceApplication {

    public static void main(String[] args) {

        SpringApplication.run(ShoppingCustcareServi
ceApplication.class, args);
    }

    /**
     * AI聊天客户端
     * @param builder
     * @return
     */
    @Bean
    public ChatClient
chatClient(ChatClient.Builder builder) {
        return builder.build();
    }
}

```

4 在 shopping\_custcare\_service 模块编写AI客服实现类

```

/**
 * AI客服服务实现类
 */
@dubboService

```

```

public class AICustCareServiceImpl
implements AICustCareService {
    @Autowired
    private ChatClient chatClient;

    @Value("${spring.ai.dashscope.prompt}")
    private String prompt;

    @Override
    public String generateResponse(String
userMessage) {
        // 添加当前时间
        PromptTemplate promptTemplate = new
PromptTemplate(prompt);
        String resolvedPrompt =
promptTemplate.render(
            Map.of("current_date",
LocalDate.now().toString())
        );
        String content =
chatClient.prompt().system(resolvedPrompt).u
ser(userMessage).call().content();
        return content;
    }
}

```

5 在 `shopping_custcare_customer_api` 模块编写AI客服控制器

```

/**
 * AI客服
 */
@RestController
@RequestMapping("/user/custcare")

```

```

public class AICustCareController {
    @DubboReference(timeout = 30000) // 指定
    30秒超时
    private AICustCareService
    aiCustCareService;

    /**
     * 回答用户问题
     * @param message 问题
     * @return 答案
     */
    @GetMapping("/answer")
    public BaseResult<String>
    answer(@RequestParam("message") String
    message){
        String answer =
        aiCustCareService.generateResponse(message);
        return BaseResult.ok(answer);
    }
}

```

## 6 测试AI功能

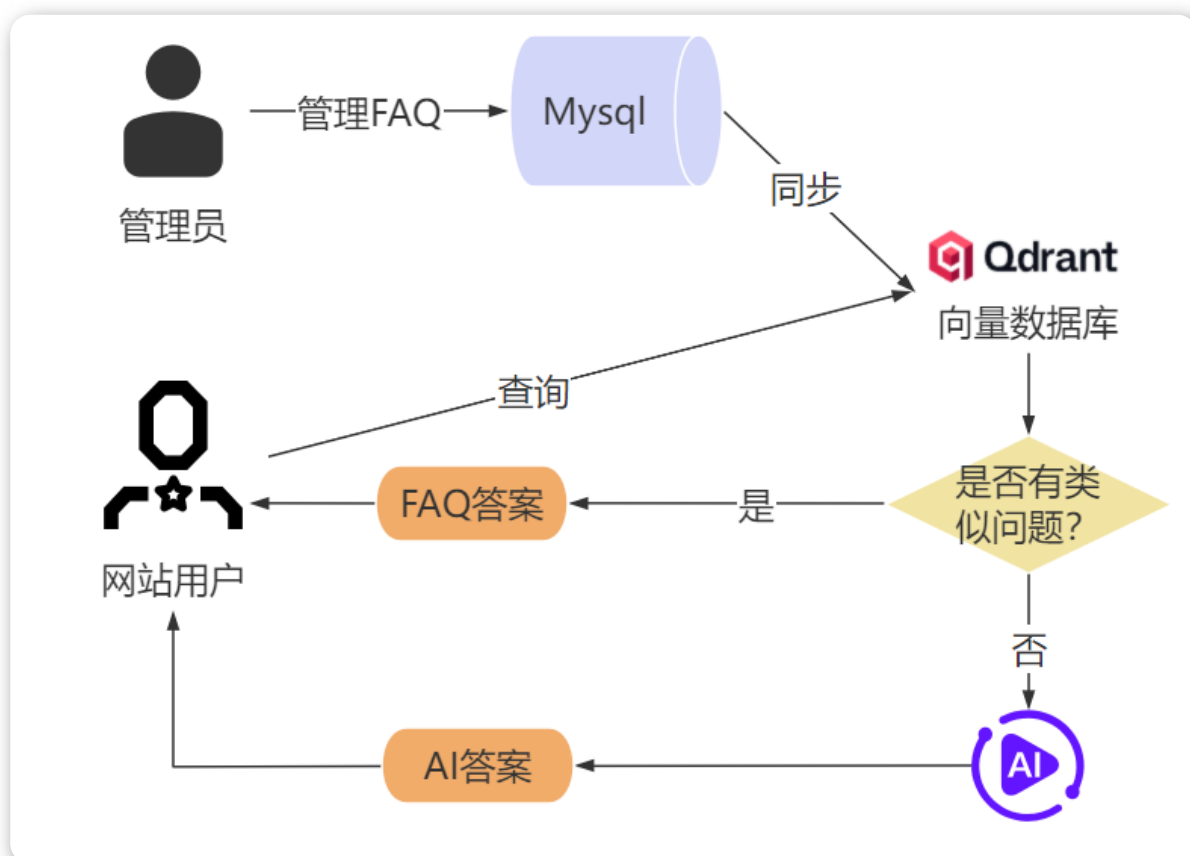
## FAQ功能

**FAQ**（Frequently Asked Questions），中文意思是“常见问题解答”。是一个组织、网站或产品中**常见问题及其答案的集合**。它旨在为用户提供快速、标准化的解答，减少重复性的客服咨询。在人工智能客服系统中，不是所有的问题我们都依赖AI回答，一些常见问题我们可以提前定义好常见问题解答，用户在问类似问题时，系统直接使用FAQ回答，这样可以节省AI的算力资源。

但是FAQ的数据不能只保存在Mysql当中，因为Mysql是无法判断问题相似度的，也就没有办法找到最相似的问题进行回答，而向量数据库可以判断问题的相似度，从而找到我们最需要的回答。所以我们需要同时把FAQ数据保存到Mysql和向量数据库Qdrant中。

FAQ功能的需求如下：

- 1 管理员向Mysql增删改查FAQ数据，并将FAQ数据同步到向量数据库Qdrant
- 2 网站用户在向客服提问时，首先在向量数据库Qdrant中寻找是否有相似问题，如果有，使用FAQ回答用户的问题，如果没有，调用AI回答用户的问题。



接下来我们就来编写FAQ功能

## 安装Qdrant

FAQ数据会保存在Mysql及Qdrant向量数据库中，用户提问时必须从Qdrant中查询，这样才能计算用户问题与FAQ问题的相似度。所以我们先来安装Qdrant：

- 1 使用Docker安装Qdrant

```
docker run -d -p 6333:6333 -p 6334:6334
qdrant/qdrant
```

② 访问Qdrant控制台: <http://ip:6333/>

③ 创建Collection保存FAQ:

```
PUT collections/common_response
{
  "vectors": {
    "size": 1536,
    "distance": "Cosine"
  }
}
```

④ 创建FAQ实体类

```
// AI客服常用回答
@Data
public class Faq implements Serializable {
    /**
     * 主键ID - 使用字符串存储UUID
     * 策略: 使用程序生成UUID, INSERT时自动设置
     */
    @TableId(value = "id", type =
    IdType.INPUT)
    private String id;

    /**
     * 分类ID
     * 1-订单问题, 2-支付问题, 3-商品问题, 4-账户
    问题, 5-其他问题
     */
    private Integer categoryId;
```

```

    /**
     * 问题内容
     */
    private String question;

    /**
     * 答案内容
     */
    private String answer;

    /**
     * 状态
     * 0-禁用，1-启用
     */
    private Integer status;

    /**
     * 使用次数
     * 记录该FAQ被匹配使用的次数
     */
    private Integer useCount;
}

```

#### 5 在Mysql创建常用回答表

```

-- -----
-- Table structure for bz_faq
-- -----

DROP TABLE IF EXISTS `bz_faq`;
CREATE TABLE `bz_faq` (
  `id` char(36) CHARACTER SET utf8 COLLATE
utf8_general_ci NOT NULL COMMENT 'UUID主键',

```



```

    `categoryId` int(11) NOT NULL COMMENT '分类
ID(1-订单问题,2-支付问题,3-商品问题,4-账户问题,5-
其他)',
    `question` varchar(500) CHARACTER SET utf8
COLLATE utf8_general_ci NOT NULL COMMENT '问
题',
    `answer` text CHARACTER SET utf8 COLLATE
utf8_general_ci NOT NULL COMMENT '答案',
    `status` tinyint(4) NULL DEFAULT 1 COMMENT
'状态(0-禁用,1-启用)',
    `useCount` int(11) NULL DEFAULT 0 COMMENT
'使用次数',
    PRIMARY KEY (`id`) USING BTREE,
    INDEX `idx_category_id` (`categoryId`)
USING BTREE,
    INDEX `idx_status` (`status`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8
COLLATE = utf8_general_ci ROW_FORMAT =
DYNAMIC;

```

```

-- -----
-- Records of bz_faq
-- -----

```

```

INSERT INTO `bz_faq` VALUES ('1b9d6bcd-bbfd-
4b2d-9b5d-ab8dfbbd4bed', 2, '支付失败怎么办?',
'支付失败请检查网络连接和账户余额, 如仍有问题请联系客
服。', 1, 0);
INSERT INTO `bz_faq` VALUES ('3f2504e0-4f89-
11d3-9a0c-0305e82c3301', 3, '商品什么时候发
货?', '一般情况下, 订单确认后24小时内发货。', 1,
0);

```

```
INSERT INTO `bz_faq` VALUES ('550e8400-e29b-41d4-a716-446655440000', 2, '支持哪些支付方式?', '我们支持微信支付、支付宝、银行卡支付等多种支付方式。', 1, 0);
INSERT INTO `bz_faq` VALUES ('6ba7b810-9dad-11d1-80b4-00c04fd430c8', 1, '如何查看订单状态?', '您可以在\"我的订单\"页面查看订单状态, 或者提供订单号我来帮您查询。', 1, 0);
INSERT INTO `bz_faq` VALUES ('8a4d0a6e-7e3a-4b5a-9f3c-1e4d5c6b7a8b', 3, '商品如何退换货?', '商品在7天内可以申请退换货, 请确保商品包装完好。', 1, 1);
INSERT INTO `bz_faq` VALUES ('a1b2c3d4-e5f6-7890-abcd-ef1234567890', 4, '如何修改个人信息?', '您可以在\"个人中心\"页面修改个人信息。', 1, 0);
INSERT INTO `bz_faq` VALUES ('d41d8cd9-8f00-3204-a980-0998ecf8427e', 4, '忘记密码怎么办?', '请使用\"忘记密码\"功能, 通过手机验证码重置密码。', 1, 0);
INSERT INTO `bz_faq` VALUES ('f47ac10b-58cc-4372-a567-0e02b2c3d479', 1, '如何取消订单?', '未付款的订单可以直接取消, 已付款的订单请联系客服处理退款。', 1, 0);

SET FOREIGN_KEY_CHECKS = 1;
```

## 管理员管理FAQ

接下来我们编写管理员管理FAQ功能:

- 1 在AI客服服务模块引入Spring ai操作Qdrant的依赖:

```

<!-- 操作qdrant向量数据库 -->
<dependency>

  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-qdrant-
store</artifactId>
  <version>1.0.0-M2</version>
</dependency>

```

## 2 在配置文件进行和操作Qdrant相关的配置

```

spring:
  # spring ai alibaba
  ai:
    vectorstore:
      qdrant:
        host: 192.168.0.99
        port: 6334
        collection-name: common_response #
collection名
        similarity-threshold: 0.7 # 匹配时的
相似度
    dashscope:
      api-key: sk-
76bcb3b45120448ea99bc6774f7a25fe
      model: qwen-max
      # 添加embedding配置
      embedding:
        api-key: sk-
76bcb3b45120448ea99bc6774f7a25fe
        # 阿里云DashScope的embedding模型
        model: text-embedding-v2
        enabled: true

```

### 3 创建Mapper

```
/**
 * FAQ答案数据访问层
 */
@Mapper
public interface FaqMapper extends
BaseMapper<Faq> {
}
```

### 4 在通用模块创建FAQ服务接口

```
/**
 * FAQ服务接口
 * 提供FAQ问答相关的业务功能
 */
public interface FaqService {
    /**
     * 分页查询FAQ答案
     * @param page 页码
     * @param size 每页大小
     * @param categoryId 分类ID
     * @return FAQ答案
     */
    Page<Faq> getFaqPage(int page, int
size, Integer categoryId);

    /**
     * 根据ID查询FAQ答案
     * @param id FAQ答案ID
     * @return FAQ答案实体
     */
    Faq getFaqById(String id);
}
```

```

/**
 * 创建FAQ答案
 *
 * @param faq FAQ答案实体
 * @return 创建的FAQ答案ID
 */
String createFaq(Faq faq);

/**
 * 更新FAQ答案
 *
 * @param faq FAQ答案实体
 */
void updateFaq(Faq faq);

/**
 * 删除FAQ答案
 *
 * @param id FAQ答案ID
 */
void deleteFaq(String id);
}

```

## 新增FAQ

接下来我们编写创建FAQ功能，在管理员创建FAQ时，需要向Mysql和Qdrant同时保存：

```

/**
 * FAQ服务实现类
 */
@Service

```

```
@Service
public class FaqServiceImpl implements
FaqService {
    @Autowired
    private FaqMapper faqMapper;

    // Spring AI提供的操作向量数据库的工具
    @Autowired
    private VectorStore vectorStore;

    /**
     * 创建FAQ答案
     *
     * @param faq FAQ答案实体
     * @return 创建的FAQ答案ID
     */
    @Override
    public String createFaq(Faq faq) {
        // 设置默认值
        if (faq.getId() == null) {

faq.setId(UUID.randomUUID().toString());
        }
        if (faq.getStatus() == null) {
            faq.setStatus(1);
        }
        if (faq.getUseCount() == null) {
            faq.setUseCount(0);
        }
        faqMapper.insert(faq);

        // 同步到向量数据库
```

```

        if (faq.getStatus() == 1) {
            insertFaqToQdrant(faq);
        }
        return faq.getId();
    }

    /**
     * 将FAQ插入到Qdrant向量数据库
     */
    private void insertFaqToQdrant(Faq faq) {
        // 构建元数据
        Map<String, Object> metadata = new
HashMap<>();
        metadata.put("faq_id", faq.getId());
        metadata.put("category_id",
faq.getCategoryId());
        metadata.put("question",
faq.getQuestion());
        metadata.put("answer",
faq.getAnswer());

        // 构建Document
        Document document = new
Document(faq.getId().toString(),
faq.getQuestion(), metadata);
        List<Document> docs = new ArrayList();
        docs.add(document);
        vectorStore.add(docs);
    }
}

```

## 删除FAQ

---

在管理员删除FAQ时，需要同时删除Mysql和Qdrant的FAQ

```
/**
 * 删除FAQ答案
 *
 * @param id FAQ答案ID
 */
@Override
public void deleteFaq(String id) {
    faqMapper.deleteById(id);
    deleteFaqVector(id);
}

/**
 * 从Qdrant删除FAQ向量
 */
private void deleteFaqVector(String faqId) {
    List<String> docs = new ArrayList();
    docs.add(faqId);
    vectorStore.delete(docs);
}
```

## 修改FAQ

---

在管理员修改FAQ时，需要同时修改Mysql和Qdrant的FAQ，但向量数据库一般不提供修改功能，所以在修改Qdrant的FAQ时，需要先删除原来的FAQ，再添加新的FAQ

```
/**
 * 更新FAQ答案
```



```

    *
    * @param faq FAQ答案实体
    */
@Override
public void updateFaq(Faq faq) {
    faqMapper.updateById(faq);
    // 同步数据到向量数据库
    if (faq.getStatus() != null &&
faq.getStatus() == 0) {
        // 删除旧文档
        deleteFaqVector(faq.getId());
    } else {
        // 删除旧文档，插入新文档
        deleteFaqVector(faq.getId());
        insertFaqToQdrant(faq);
    }
}
}

```

## 查询FAQ

管理员查询FAQ时是直接从Mysql中进行查询，接下来我们就编写该功能：

```

/**
 * 分页查询FAQ答案
 * @param page      页码
 * @param size      每页大小
 * @param categoryId 分类ID
 * @return FAQ答案分页结果
 */
@Override

```

```

public Page<Faq> getFaqPage(int page, int size,
Integer categoryId) {
    QueryWrapper<Faq> wrapper = new
QueryWrapper<>();
    if (categoryId != null) {
        wrapper.eq("categoryId", categoryId);
    }
    return faqMapper.selectPage(new Page(page,
size), wrapper);
}

/**
 * 根据ID查询FAQ答案
 *
 * @param id FAQ答案ID
 * @return FAQ答案实体
 */
@Override
public Faq getFaqById(String id) {
    return faqMapper.selectById(id);
}

```

## 管理员管理FAQ控制器

接下来我们在 `shopping_manager_api` 模块添加管理FAQ控制器：

```

/**
 * FAQ管理控制器（管理端）
 * 提供FAQ的增删改查功能
 */
@RestController

```

```
@RequestMapping("/faq")
public class FaqController {
    @DubboReference
    private FaqService faqService;

    /**
     * 分页查询已启用的FAQ
     * @param page 页码
     * @param size 每页条数
     * @param categoryId 分类ID（可选）
     * @return 查询结果
     */
    @GetMapping("/search")
    public BaseResult<Page<Faq>>
search(@RequestParam(defaultValue = "1") int
page,

@RequestParam(defaultValue = "10") int size,

@RequestParam(required = false) Integer
categoryId) {
        Page<Faq> faqPage =
faqService.getFaqPage(page, size, categoryId);
        return BaseResult.ok(faqPage);
    }

    /**
     * 根据ID查询FAQ
     * @param id FAQ ID
     * @return 查询结果
     */
    @GetMapping("/findById")
```

```
    public BaseResult<Faq>
findById(@RequestParam String id) {
    Faq faq = faqService.getFaqById(id);
    return BaseResult.ok(faq);
}

/**
 * 新增FAQ
 * @param faq FAQ实体
 * @return 操作结果
 */
@PostMapping("/add")
public BaseResult<String> add(@RequestBody
Faq faq) {
    String id = faqService.createFaq(faq);
    return BaseResult.ok(id);
}

/**
 * 修改FAQ
 * @param faq FAQ实体
 * @return 操作结果
 */
@PutMapping("/update")
public BaseResult update(@RequestBody Faq
faq) {
    faqService.updateFaq(faq);
    return BaseResult.ok();
}

/**
 * 删除FAQ
```

```

    * @param id FAQ ID
    * @return 操作结果
    */
    @DeleteMapping("/delete")
    public BaseResult delete(@RequestParam
String id) {
        faqService.deleteFaq(id);
        return BaseResult.ok();
    }
}

```

## 同步Mysql数据到FAQ

接下来我们编写一个方法，将Mysql中的FAQ数据同步到Qdrant中：

### ① 修改FaqServiceImpl

```

/**
 * 同步FAQ数据到向量数据库
 */
public void syncToQdrant() {
    QueryWrapper<Faq> wrapper = new
QueryWrapper<>();
    wrapper.eq("status", 1);
    List<Faq> faqs =
faqMapper.selectList(wrapper);
    for (Faq faq : faqs) {
        insertFaqToQdrant(faq);
    }
}
}

```

### ② 编写测试方法

```

@SpringBootTest
class
ShoppingCustcareServiceApplicationTests {
    @Autowired
    private FaqServiceImpl faqService;

    @Test
    void testSyncToQdrant(){
        faqService.syncToQdrant();
    }
}

```

## 用户向Qdrant查询FAQ

接下来我们编写用户查询FAQ的功能，由于用户是输入一个问题，依靠该问题查询是否有已经配置好的相似的FAQ问题。所以用户是在Qdrant中进行查询，接下来我们就来编写用户查询FAQ功能：

### ① 修改FaqService

```

/**
 * 使用向量搜索智能匹配最佳答案
 *
 * @param question 用户问题
 * @return 匹配的FAQ答案，如果没有匹配返回null
 */
Faq findBestAnswer(String question);

```

### ② 修改FaqServiceImpl

```

@Value("${spring.ai.vectorstore.qdrant.similarity-threshold}")
private double similarityThreshold;

```

```
/**
 * 智能匹配最佳答案
 *
 * @param question 用户问题
 * @return 匹配的FAQ答案
 */
@Override
public Faq findBestAnswer(String question) {
    // 向量搜索FAQ答案
    Faq vectorResult =
findByVectorSearch(question);
    // 增加FAQ使用次数
    if (vectorResult != null) {

incrementUseCount(vectorResult.getId());
    }
    return vectorResult;
}

/**
 * 使用向量搜索匹配FAQ答案
 */
private Faq findByVectorSearch(String
question) {
    // 构建搜索请求
    SearchRequest searchRequest =
SearchRequest
        .query(question)
        .withTopK(1)
```

```

.withSimilarityThreshold(similarityThreshold
);
    // 执行搜索
    List<Document> results =
vectorStore.similaritySearch(searchRequest);
    if (!results.isEmpty()) {
        Document result = results.get(0);
        String faqId =
result.getId().toString();
        return faqMapper.selectById(faqId);
    }
    return null;
}

/**
 * 增加FAQ使用次数
 *
 * @param id FAQ答案ID
 */
private void incrementUseCount(String id) {
    Faq faq = faqMapper.selectById(id);
    faq.setUseCount(faq.getUseCount() + 1);
    faqMapper.updateById(faq);
}

```

### 3 修改AICustCareServiceImpl

```

/**
 * AI客服服务实现类
 */
@dubboService

```



```
public class AICustCareServiceImpl
implements AICustCareService {
    @Autowired
    private ChatClient chatClient;

    @Autowired
    private FaqServiceImpl faqService;

    @Value("${spring.ai.dashscope.prompt}")
    private String prompt;

    @Override
    public String generateResponse(String
userMessage) {
        // 添加当前时间
        PromptTemplate promptTemplate = new
PromptTemplate(prompt);
        String resolvedPrompt =
promptTemplate.render(
            Map.of("current_date",
LocalDate.now().toString())
        );
        // 先尝试FAQ回答，如果没有答案再使用AI
        Faq bestAnswer =
faqService.findBestAnswer(userMessage);
        if (bestAnswer != null) {
            return bestAnswer.getAnswer();
        } else {
            String content =
chatClient.prompt().system(resolvedPrompt).u
ser(userMessage).call().content();
            return content;
        }
    }
}
```

```
    }  
  }  
}
```

- 4 分别测试用户提出FAQ问题时与普通问题时的响应

## 过滤敏感词

在和AI聊天时，我们要对一些敏感词进行过滤，比如政治敏感词，色情/涉黄词，暴力/脏话词，广告/营销词，违法词。AI只会回答与敏感词无关的提问。所以我们需要在AI客服中引入过滤敏感词的功能：

- 1 引入敏感词过滤依赖

```
<!-- Sensitive word 库 -->  
<dependency>  
  <groupId>com.github.houbb</groupId>  
  <artifactId>sensitive-word</artifactId>  
  <version>0.21.0</version>  
</dependency>
```

- 2 在AI客服服务模块创建敏感词配置类

```
@Configuration  
public class SensitivewordConfig {  
    @Bean  
    public SensitivewordBs sensitivewordBs()  
    {  
        return SensitivewordBs.newInstance()  
            // 黑名单：默认  
  
        .wordDeny(wordDenys.chains(wordDenys.defaults()  
s()))
```

```

        // 白名单：默认
        .wordAllow(wordAllows.chains(wordAllows.defaults()))

        // 忽略大小写
        .ignoreCase(true)
        // 忽略全角/半角
        .ignoreWidth(true)
        // 启用邮箱检查
        .enableEmailCheck(true)
        // 启用URL检查
        .enableUrlCheck(true)
        // 启用数字检查
        .enableNumCheck(true)
        // 初始化
        .init();
    }
}

```

### ③ 修改AI客服服务实现类

```

/**
 * AI客服服务实现类
 */
@Service
public class AICustCareServiceImpl
implements AICustCareService {
    @Autowired
    private ChatClient chatClient;
    @Autowired
    private SensitiveWordBs sensitiveWordBs;
    @Autowired
    private FaqServiceImpl faqService;
}

```

```
@Value("${spring.ai.dashscope.prompt}")
private String prompt;

@Override
public String generateResponse(String
userMessage) {
    // 添加当前时间
    PromptTemplate promptTemplate = new
PromptTemplate(prompt);
    String resolvedPrompt =
promptTemplate.render(
        Map.of("current_date",
LocalDate.now().toString())
    );
    // 1.检查敏感词
    String error = check(userMessage);
    if (error != null) {
        return error;
    }
    // 2.先尝试FAQ回答
    Faq bestAnswer =
faqService.findBestAnswer(userMessage);
    if (bestAnswer != null) {
        return bestAnswer.getAnswer();
    }
    // 3.如果FAQ没有答案使用AI回答
    String content =
chatClient.prompt().system(resolvedPrompt).u
ser(userMessage).call().content();
    return content;
}
```

```

/**
 * 检查文本是否包含敏感词
 * @param text 文本
 * @return 错误信息，如果没有错误返回null
 */
private String check(String text) {
    if (text == null ||
text.trim().isEmpty()) {
        return "内容不能为空";
    }
    if (sensitiveWords.contains(text))
{
        return "内容包含敏感词汇，这个问题我
暂时无法回答，让我们换个话题再聊聊吧！";
    }
    return null;
}
}

```

#### ④ 测试过滤敏感词功能

## 管理敏感词黑/白名单

我们目前使用的是sensitive-word的默认敏感词，而在实际项目中，每个项目都有自己特有的敏感词，以及敏感词白名单，所以我们需要有管理自定义敏感词黑/白名单的功能：

#### ① 数据库创建敏感词黑/白名单表：

```

CREATE TABLE bz_sensitive_word (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    word VARCHAR(255) NOT NULL COMMENT '敏感词',
    type ENUM('deny', 'allow') DEFAULT 'deny' COMMENT '类型: deny-黑名单, allow-白名单'
);

INSERT INTO bz_sensitive_word (word, type)
VALUES
('假货', 'deny'),
('刷单', 'deny'),
('正品', 'allow'),
('退货', 'allow');

```

## 2 创建敏感词实体类

```

@Data
public class SensitiveWord implements
Serializable{
    @TableId(type = IdType.AUTO)
    private Long id;
    private String word;
    private String type; // "deny" 或 "allow"
}

```

## 3 创建敏感词Mapper

```
/**
 * 敏感词黑/白名单 Mapper接口
 */
@Mapper
public interface SensitivewordMapper extends
BaseMapper<Sensitiveword> {
}
```

#### 4 创建敏感词服务接口

```
/**
 * 敏感词服务接口
 */
public interface SensitivewordService {

    /**
     * 添加敏感词
     * @param sensitiveword 敏感词对象
     */
    void addSensitiveword(Sensitiveword
sensitiveword);

    /**
     * 删除敏感词
     * @param id 敏感词ID
     */
    void deleteSensitiveword(Long id);

    /**
     * 分页查询敏感词
     * @param page 页码
     * @param size 每页大小
     * @param word 关键词（可选，模糊匹配）
     */
}
```

```

        * @param type 敏感词类型 ("deny" 或
"allow")
        * @return 分页结果
    */
    Page<Sensitiveword>
    findSensitivewords(int page, int size,
String word ,String type);
}

```

## 5 创建敏感词服务实现类

```

/**
 * 敏感词服务实现类
 */
@Service
public class SensitivewordServiceImpl
implements SensitivewordService {
    @Autowired
    private SensitivewordMapper
    sensitivewordMapper;

    /**
     * 添加敏感词
     * @param sensitiveword 敏感词对象
     */
    @Override
    public void
    addSensitiveword(Sensitiveword
    sensitiveword) {

        sensitivewordMapper.insert(sensitiveword);
    }

    /**

```



```

        * 删除敏感词
        * @param id 敏感词ID
        */
@Override
public void deleteSensitiveWord(Long id)
{
    sensitiveWordMapper.deleteById(id);
}

/**
 * 分页查询敏感词
 * @param page 页码
 * @param size 每页大小
 * @param word 关键词（可选，模糊匹配）
 * @param type 敏感词类型 ("deny" 或
"allow")
 * @return
 */
@Override
public Page<SensitiveWord>
findSensitiveWords(int page, int size,
String word ,String type) {
    QueryWrapper<SensitiveWord> wrapper
= new QueryWrapper<>();
    if (word != null && !word.isEmpty())
    {
        wrapper.like("word", word);
    }
    if (type != null && !type.isEmpty())
    {
        wrapper.eq("type", type);
    }
}

```

```

        return
    sensitivewordMapper.selectPage(new
    Page(page, size), wrapper);
    }
}

```

## 6 创建敏感词管理控制器

```

/**
 * 敏感词管理控制器
 */
@RestController
@RequestMapping("/sensitiveWord")
public class SensitiveWordController {

    @DubboReference
    private SensitiveWordService
    sensitivewordService;

    @PostMapping("/add")
    public BaseResult
    addSensitiveWord(@RequestBody SensitiveWord
    sensitiveword) {

        sensitivewordService.addSensitiveWord(sensi
        tiveword);
        return BaseResult.ok();
    }

    @DeleteMapping("/delete")
    public BaseResult
    deleteSensitiveWord(Long id) {

```

```

        sensitivewordService.deleteSensitiveword(id
    );

        return BaseResult.ok();
    }

    @GetMapping("/search")
    public BaseResult<Page<Sensitiveword>>
search(@RequestParam(defaultValue = "1") int
page,

        @RequestParam(defaultValue = "10") int
size,

        @RequestParam(required = false) String
word,

        @RequestParam(required = false) String
type) {
        Page<Sensitiveword> sensitivewords =
sensitivewordService.findSensitivewords(page
, size,word,type);
        return
BaseResult.ok(sensitivewords);
    }
}

```

## 7 测试管理敏感词黑/白名单功能

### 将敏感词黑/白名单应用于AI客服

接下来我们将自定义敏感词黑/白名单应用于AI客服

① 修改敏感词配置类，添加自定义黑/白名单

```
@Configuration
public class SensitivewordConfig {
    @Autowired
    private SensitivewordMapper
    sensitivewordMapper;

    @Bean
    public SensitivewordBs sensitivewordBs()
    {
        return
        SensitivewordBs.newInstance()
            // 黑名单：默认+自定义

        .wordDeny(wordDenys.chains(wordDenys.defaults(), new IWordDeny() {
            @Override
            public
            List<String> deny() {

                QueryWrapper<Sensitiveword> wrapper = new
                Querywrapper<>();

                wrapper.eq("type", "deny");

                return
                sensitivewordMapper.selectList(wrapper)

                .stream()

                .map(Sensitiveword::getword)

                .toList();
            }
        }));
    }
}
```

```

    }
    })
)
// 白名单：默认+自定义

.wordAllow(wordAllows.chains(wordAllows.defaults(), new IwordAllow() {
    @Override
    public
    List<String> allow() {

        QueryWrapper<SensitiveWord> wrapper = new
        QueryWrapper<>();

        wrapper.eq("type", "allow");

        return
        sensitivewordMapper.selectList(wrapper)

        .stream()

        .map(SensitiveWord::getWord)

        .toList();

    }
    })
)
// 忽略大小写
.ignoreCase(true)
// 忽略全角/半角
.ignoreWidth(true)
// 启用邮箱检查
.enableEmailCheck(true)

```

```

        // 启用URL检查
        .enableUrlCheck(true)
        // 启用数字检查
        .enableNumCheck(true)
        // 初始化
        .init();
    }
}

```

- 2 修改敏感词服务实现类，在新增删除敏感词黑白名单后，同步到配置中

```

/**
 * 敏感词服务实现类
 */
@Service
public class SensitiveWordServiceImpl
implements SensitiveWordService {
    @Autowired
    private SensitiveWordMapper
sensitiveWordMapper;
    @Autowired
    private SensitiveWordBs sensitiveWordBs;
    // 敏感词配置类

    /**
     * 添加敏感词
     * @param sensitiveWord 敏感词对象
     */
    @Override
    public void
addSensitiveWord(SensitiveWord
sensitiveWord) {

```

```
sensitivewordMapper.insert(sensitiveword);
    // 刷新配置
    if
(sensitiveword.getType().equals("deny")){
        // 添加到黑名单

    sensitivewordBs.addword(sensitiveword.getwo
rd());
        }else if
(sensitiveword.getType().equals("allow")){
            // 添加到白名单

    sensitivewordBs.addwordAllow(sensitiveword.
getword());
        }
    }

    /**
     * 删除敏感词
     * @param id 敏感词ID
     */
    @Override
    public void deleteSensitiveword(Long id)
    {
        Sensitiveword sensitiveword =
sensitivewordMapper.selectById(id);
        sensitivewordMapper.deleteById(id);
        // 刷新配置
        if
(sensitiveword.getType().equals("deny")){
            // 删除黑名单
```

```

        sensitivewordBs.removeWord(sensitiveword.getword());
    }else if
(sensitiveword.getType().equals("allow")){
        // 删除白名单

        sensitivewordBs.removeWordAllow(sensitiveword.getword());
    }
}

/**
 * 分页查询敏感词
 * @param page 页码
 * @param size 每页大小
 * @param type 敏感词类型 ("deny" 或
"allow")
 * @return
 */
@Override
public Page<Sensitiveword>
findSensitivewords(int page, int size,
String type) {
    QueryWrapper<Sensitiveword> wrapper
= new QueryWrapper<>();
    if (type != null && !type.isEmpty())
    {
        wrapper.eq("type", type);
    }
}

```



```
        return  
        sensitivewordMapper.selectPage(new  
        Page(page, size), wrapper);  
    }  
  
}
```

### ③ 测试管理敏感词黑/白名单功能

## AI查询订单功能

客户经常会使用客服帮忙操作订单，比如订单查询、订单取消等操作，这里我们开发AI客服查询订单的功能，即让AI调用我们系统的方法查询订单并向客户返回。

### ① 编写订单查询服务

```
@Component  
public class OrderQueryService implements  
Function<OrderQueryService.Request, OrderQuer  
yService.Response> {  
    @DubboReference  
    private OrdersService ordersService;  
  
    /**  
     * 应用  
     * @param  
     * @return  
     */  
    @Override  
    public Response apply(Request request) {  
        Long userId = request.userId();  
        Integer status = request.status();
```

```

        int size = request.size() != null ?
request.size() : 1; // 默认 1 条
        List<Orders> orders =
ordersService.findUserOrders(userId,status);
        List<Orders> subList =
orders.subList(0, size < orders.size() ?
size : orders.size());
        return new Response(subList);
    }

    /**
     * 输入
     * @param userId 用户ID
     * @param size 查询条数
     * @param status 订单状态
     */
    public record Request(Long userId ,
Integer size,Integer status){}

    /**
     * 输出
     * @param orders 订单信息集合
     */
    public record Response(List<Orders>
orders){}

}

```

## 2 注册订单查询服务

```

@Configuration
public class FunctionCallConfig {
    @Bean
    @Description("查询当前用户的订单信息，输入订单状态（1=待支付，2=已完成）和查询条数（默认为1），用户ID从prompt获取")
    public
    Function<OrderQueryService.Request,OrderQueryService.Response> getOrderQueryService(){
        return new OrderQueryService();
    }
}

```

### 3 修改AI客服实现类

```

@Override
public String generateResponse(String
userMessage,Long userId) {
    // 添加当前时间
    //          PromptTemplate promptTemplate
    = new PromptTemplate(prompt);
    //          String resolvedPrompt =
    promptTemplate.render(
        //          Map.of("current_date",
        LocalDate.now().toString())
        //          );
    // 1.检查敏感词
    String error = check(userMessage);
    if (error != null) {
        return error;
    }
    // 2.先尝试FAQ回答
}

```

```

        Faq bestAnswer =
faqService.findBestAnswer(userMessage);
        if (bestAnswer != null) {
            return bestAnswer.getAnswer();
        }
        // 3.如果FAQ没有答案使用AI回答
        String prompt = ""
            你是百战电商的AI客服小白，专注于帮助用户查询
            订单信息。
            用户可能会用自然语言请求订单查询，如“帮我查
            询最近的订单”或“我的待支付订单”。
            请优先使用工具函数
            `getOrderQueryService` 处理订单查询请求。
            仅当用户明确要求其他操作或工具无法处理时，才
            直接回复。
            当前用户: {user_id}
            """;
        // 添加当前时间
        PromptTemplate promptTemplate = new
        PromptTemplate(prompt);
        String resolvedPrompt =
        promptTemplate.render(
            Map.of("user_id", userId)
        );
        String content =
        chatClient.prompt().system(resolvedPrompt).u
        ser(userMessage).call().content();
        return content;
    }

```

#### ④ 修改AI聊天客户端配置

```
@SpringBootApplication
```

```

@EnableDiscoveryClient
@EnableDubbo
@RefreshScope
@MapperScan("com.itbaizhan.shopping_custcare
_service.mapper")
public class
ShoppingCustcareServiceApplication {

    public static void main(String[] args) {

        SpringApplication.run(ShoppingCustcareServi
ceApplication.class, args);
    }

    /**
     * AI聊天客户端
     * @param builder
     * @return
     */
    @Bean
    public ChatClient
chatClient(ChatClient.Builder builder) {
        return builder

        .defaultFunctions("getOrderQueryService")
            .build();
    }
}

```

#### 5 修改AI聊天控制器

```

/**

```

```

    * AI客服
    */
@RestController
@RequestMapping("/user/custcare")
public class AICustCareController {
    @DubboReference(timeout = 30000) // 指定
    30秒超时
    private AICustCareService
    aiCustCareService;
    @DubboReference
    private ShoppingUserService
    shoppingUserService;

    /**
     * 回答用户问题
     * @param message 问题
     * @return 答案
     */
    @GetMapping("/answer")
    public BaseResult<String>
    answer(@RequestParam("message") String
    message, @RequestHeader("Authorization")
    String authorization){
        String token =
    authorization.replace("Bearer ", "");
        Long userId = null;
        try {
            ShoppingUser loginUser =
    shoppingUserService.getLoginUser(token);
            userId = loginUser.getId();
        }catch (Exception e){}
    }
}

```

```
        String answer =
aiCustCareService.generateResponse(message, u
serId);

        return BaseResult.ok(answer);
    }
}
```

- 6 启动订单服务，测试订单查询功能：帮我查询最新的订单

## AI搜索商品功能

我们的AI客服除了可以帮助用户查询订单，还可以搜索商品，接下来我们就来编写AI搜索商品功能：

- 1 编写商品搜索服务

```
@Component
public class GoodsQueryService implements
Function<GoodsQueryService.Request, GoodsQuer
yService.Response> {
    @DubboReference
    private SearchService searchService;

    /**
     * 应用
     * @param
     * @return
     */
    @Override
    public GoodsQueryService.Response
apply(Request request) {
        Integer size = request.size == null
? 1 : request.size; // 默认 1 条
```

```

        String keyword = request.keyword ==
null ? "" : request.keyword; // 默认空
        GoodsSearchParam goodsSearchParam =
new GoodsSearchParam();
        goodsSearchParam.setPage(1);
        goodsSearchParam.setSize(size);

        goodsSearchParam.setKeyword(keyword);
        List<GoodsES> goodsESList =
searchService.search(goodsSearchParam).getGo
odsPage().getRecords();
        return new Response(goodsESList);
    }

    /**
     * 输入
     * @param keyword 关键字
     * @param size 查询条数
     */
    public record Request(String
keyword,Integer size){}

    /**
     * 输出
     * @param goodsESList 商品信息集合
     */
    public record Response(List<GoodsES>
goodsESList){}
}

```

## 2 注册商品查询服务

```
@Configuration
```



```

public class FunctionCallConfig {
    @Bean
    @Description("查询当前用户的订单信息，输入订单状态（1=待支付，2=已完成）和查询条数（默认为1），用户ID从prompt获取")
    public
Function<OrderQueryService.Request,OrderQueryService.Response> getOrderQueryService(){
        return new OrderQueryService();
    }

    @Bean
    @Description("查询商品信息，输入商品关键字和查询条数（默认为1）")
    public
Function<GoodsQueryService.Request,GoodsQueryService.Response> getGoodsQueryService() {
        return new GoodsQueryService();
    }
}

```

### 3 修改AI聊天客户端配置

```

@SpringBootApplication
@EnableDiscoveryClient
@EnableDubbo
@RefreshScope
@MapperScan("com.itbaizhan.shopping_custcare_service.mapper")
public class
ShoppingCustcareServiceApplication {

    public static void main(String[] args) {

```

```

        SpringApplication.run(ShoppingCustcareServiceApplication.class, args);
    }

    /**
     * AI聊天客户端
     * @param builder
     * @return
     */
    @Bean
    public ChatClient
    chatClient(ChatClient.Builder builder) {
        return builder

        .defaultFunctions("getOrderQueryService", "getGoodsQueryService")
            .build();
    }
}

```

#### ④ 修改AI客服实现类

```

@Override
public String generateResponse(String
userMessage, Long userId) {
    // 1.检查敏感词
    String error = check(userMessage);
    if (error != null){
        return error;
    }
    // 2.先尝试FAQ回答

```

```
    Faq bestAnswer =  
faqService.findBestAnswer(userMessage);  
    if (bestAnswer != null) {  
        return bestAnswer.getAnswer();  
    }
```

// 3.如果没有FAQ答案,调用AI模型回答

```
prompt += ""
```

你是一个专业的电商客服助手，专门处理订单查询和商品查询请求。请严格按照以下规则执行：

1. **\*\*意图识别\*\***:

- 当用户提及"订单"、"我的订单"、"购买记录"、"待支付"、"已发货"、"最近订单"等关键词时，识别为订单查询意图
- 当用户提及"商品"、"查找"、"搜索"、"手机"、"衣服"、"价格"、"库存"等关键词时，识别为商品查询意图

2. **\*\*服务调用\*\***:

- 订单查询意图 → 调用  
`getOrderQueryService` 方法
- 商品查询意图 → 调用  
`getGoodsQueryService` 方法

请确保准确识别用户意图并正确调用相应服务，提取所有可用参数填充到函数调用中，用户ID从prompt获取。

仅当用户明确要求其他操作或工具无法处理时，才直接回复。

当前用户的id: {user\_id}

```
"";
```

// 添加当前时间用户Id

```
    PromptTemplate promptTemplate = new  
PromptTemplate(prompt);  
    String resolvedPrompt =  
promptTemplate.render(
```

```

        Map.of("current_date",
LocalDate.now().toString(),
            "user_id",userId)
    );
    // AI回答
    String content =
chatClient.prompt().system(resolvedPrompt).u
ser(userMessage).call().content();
    return content;
}

```

- ⑤ 启动商品查询服务，测试商品查询功能：帮我查询iphone手机

## 对话记忆功能

我们的AI客服不仅可以回答当前的问题，还需要在回答问题的时候结合之前的提问和回答，这就需要AI客服在对话中记住先前的对话内容。而Spring AI Alibaba提供了基于内存存储的对话记忆。接下来我们就在AI客服中引入对话记忆功能：

- ① 在启动类配置对话记忆

```

@SpringBootApplication
@EnableDiscoveryClient
@EnableDubbo
@RefreshScope
@MapperScan("com.itbaizhan.shopping_custcare
_service.mapper")
public class
ShoppingCustcareServiceApplication {

    public static void main(String[] args) {

```

```

    SpringApplication.run(ShoppingCustcareServiceApplication.class, args);
}

/**
 * 对话记忆
 */
@Bean
public ChatMemory chatMemory(){
    return new InMemoryChatMemory();
}

/**
 * AI聊天客户端
 */
@Bean
public ChatClient
chatClient(ChatClient.Builder
builder, ChatMemory chatMemory) {
    return builder
        // 函数调用

        .defaultFunctions("getOrderQueryService", "getGoodsQueryService")
        // 对话记忆
        .defaultAdvisors(new
MessageChatMemoryAdvisor(chatMemory))
        .build();
}
}

```

## 2 在AI客服实现类加入对话记忆

```
@Override
public String generateResponse(String
userMessage, Long userId) {
    // 1.检查敏感词
    String error = check(userMessage);
    if (error != null){
        return error;
    }
    // 2.先尝试FAQ回答
    Faq bestAnswer =
faqService.findBestAnswer(userMessage);
    if (bestAnswer != null) {
        return bestAnswer.getAnswer();
    }
    // 3.如果没有FAQ答案,调用AI模型回答
```

```
prompt += """
```

你是一个专业的电商客服助手，专门处理订单查询和商品查询请求。请严格按照以下规则执行：

### 1. \*\*意图识别\*\*：

- 当用户提及"订单"、"我的订单"、"购买记录"、"待支付"、"已发货"、"最近订单"等关键词时，识别为订单查询意图

- 当用户提及"商品"、"查找"、"搜索"、"手机"、"衣服"、"价格"、"库存"等关键词时，识别为商品查询意图

### 2. \*\*服务调用\*\*：

- 订单查询意图 → 调用`getOrderQueryService`方法
- 商品查询意图 → 调用`getGoodsQueryService`方法

请确保准确识别用户意图并正确调用相应服务，提取所有可用参数填充到函数调用中，用户ID从prompt获取。

仅当用户明确要求其他操作或工具无法处理时，才直接回复。

```
    当前用户的id: {user_id}
    """;

    // 添加当前时间用户Id
    PromptTemplate promptTemplate = new
    PromptTemplate(prompt);
    String resolvedPrompt =
    promptTemplate.render(
        Map.of("current_date",
        LocalDate.now().toString(),
            "user_id",userId)
    );
    // AI回答
    String content = chatClient
        .prompt()
        .system(resolvedPrompt)
        .user(userMessage)
        // 记忆管理
        .advisors(spec -> spec

        .param(CHAT_MEMORY_CONVERSATION_ID_KEY,
        userId) // 使用用户ID作为会话ID

        .param(CHAT_MEMORY_RETRIEVE_SIZE_KEY,10)) //
        检索最近10条消息
        .call()
        .content();
    return content;
```

}

- 3 测试对话记忆功能：我的上一个问题是什么。



